데이터베이스

데이터베이스 관리 시스템

오병우

컴퓨터공학부

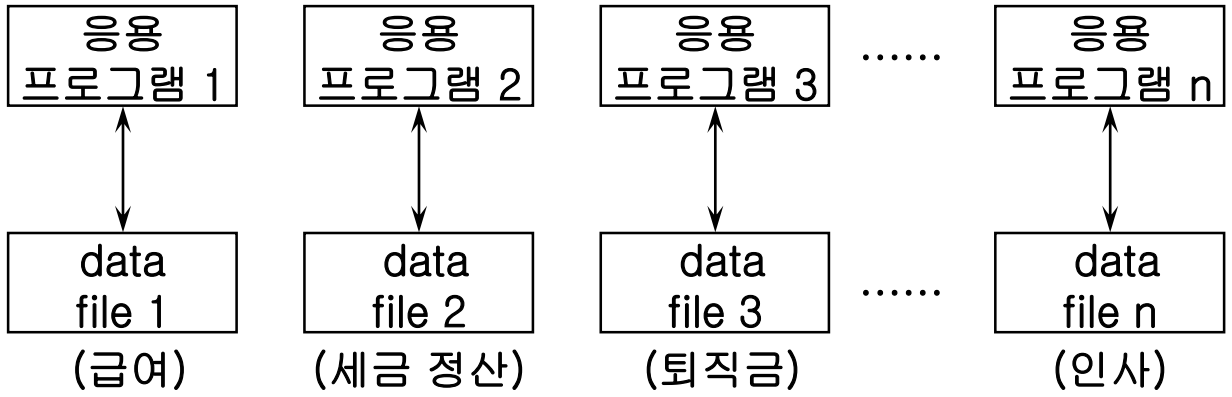
kit 국립금오공과대학교
Kumoh National Institute of Technology

File (파일) System

- 불일치성 (Inconsistency)
  - 각 응용 프로그램은 각자 자신만의 파일 사용 (Cobol 언어 사용하던 시절)
    - 광범위하게 분산되어서 제어하기 어려움
- File 중심 자료 처리 시스템의 한계
  - 응용 프로그램의 논리적 파일 구조는 직접 물리적 파일 구조로 구현
  - 응용 프로그램에 물리적 데이터 구조에 대한 접근 방법이 구현되어야 함
  - 데이터 파일의 공용이 불가능
- 문제점
  - 데이터 종속성 (dependency) we can say gpu is independent bc when u trying to change gpu for some kind of fuction to add ur pc, there is no necessary to install os again
  - 데이터 중복성 (data redundancy)

File System

- File 시스템에서 응용 프로그램과 file과의 관계



데이터 종속성

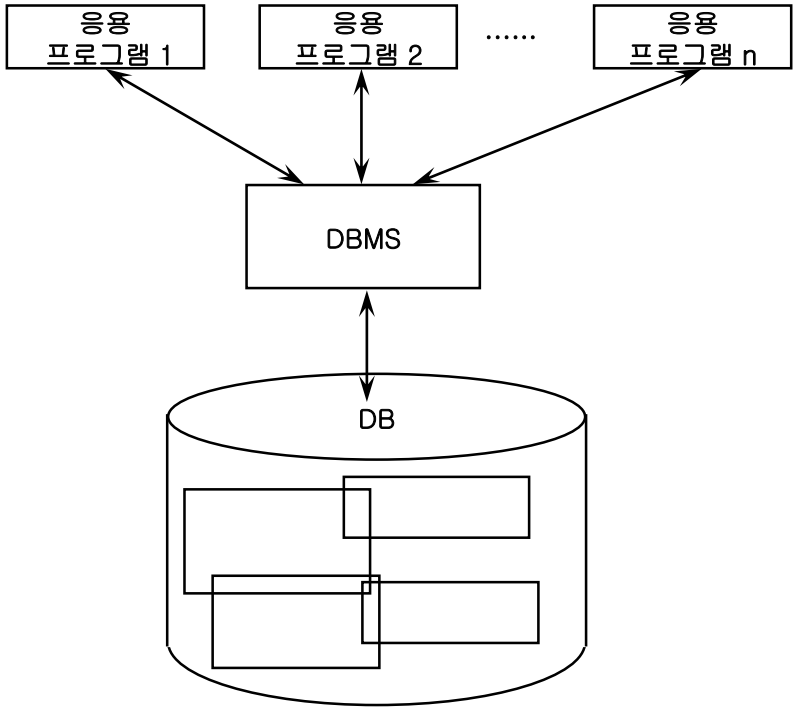
- 데이터 종속성
  - 저장 구조 또는 접근 전략 변경
    - 응용 프로그램에 영향을 끼침
  - 데이터 구조와 접근할 방법을 응용 프로그램이 로직과 코드로 구현
- 데이터 독립성 필요
  - 동일한 데이터에 대해 두 응용 프로그램이 다른 뷰를 가지고 있을 수 있음
    - Field: decimal or binary => DBMS: conversion
  - DBA는 응용 프로그램에 영향을 주지 않으면서도 마음대로 저장 구조 및 접근 전략 변경할 수 있어야 함
    - 새로운 데이터 추가
    - 새로운 표준 채택
    - 응용 프로그램 우선 순위 변경
    - 새로운 타입의 저장 장치 사용

데이터 중복성

- Data Redundancy (데이터 중복성)
  - 한 시스템 내에 같은 내용의 데이터가 중복되어 저장 관리
- 문제점
  - Consistency(일관성): 중복되는 데이터 간의 불일치(inconsistency)
  - Security(보안성): 중복되는 데이터를 같은 수준으로 접근 통제
  - Economics(경제성): 중복되는 저장 공간 및 중복 수정
  - Integrity(무결성): 의미적 오류(강좌에는 강사가 있어야 한다.)

데이터베이스 관리 시스템 (DBMS)

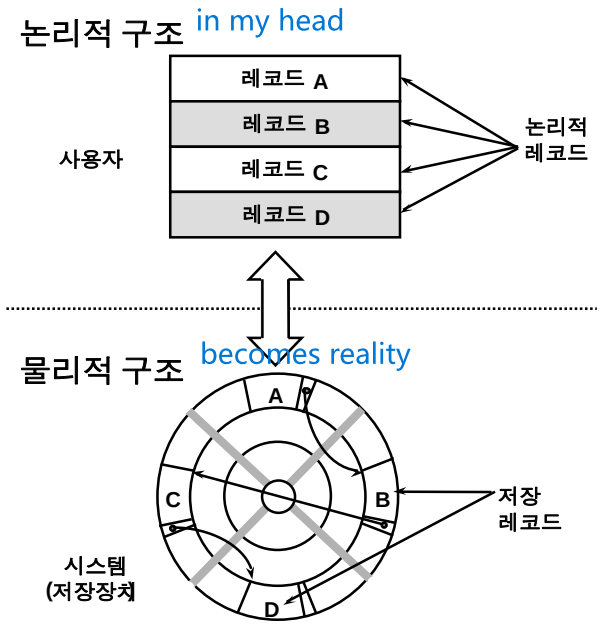
- Database Management System (DBMS) or database manager
  - A layer of software between the physical database and the user
  - Handle all requests from users for access to the database
  - 응용 프로그램과 데이터 사이의 중재자로서 모든 응용 프로그램들이 데이터베이스를 공용할 수 있게 관리해 주는 소프트웨어 시스템
- DBMS의 필수 기능
  - Data Definition: create, drop, alter
  - Data Manipulation: insert, select, delete, update
  - Data Control: grant, revoke



Data Definition

- 정의 (definition) 기능
  - 하나의 저장 구조로 여러 사용자의 요구를 지원할 수 있도록 데이터를 조직하는 기능

  - 정의 기능의 요건
    - 데이터의 논리적 구조를 명세
    - 데이터의 물리적 구조 명세
    - 물리적/논리적 Mapping (사상) 명세



Data Manipulation

- 조작 (manipulation) 기능
  - 사용자- 데이터베이스 간의 인터페이스
  - 체계적 데이터 처리 지원 능력 (CRUD)
    creat read update delete
    - Create: 삽입
    - Read or Retrieve: 검색
    - Update: 갱신
    - Delete: 삭제
  - 조작 기능의 요건
    - 사용이 쉽고 자연스러운 도구
    - 원하는 연산의 명세 가능
    - 효율적인 접근
- 결함이 없는 (integrity, 무결성) 상태로부터 DML 연산을 통해 (다시) 결함이 없는 상태로 변화

Data Control

- 제어 (control) 기능
  - 데이터의 정확성과 보안성을 유지하는 기능
  - 제어 기능의 요건
    - 무결성 (integrity) 유지
    - 보안, 권한 검사
    - 병행 수행 제어 (concurrency control)
- Transaction 처리
  - Data Transaction Language로 별도로 분류하기도 함

DBMS의 장단점

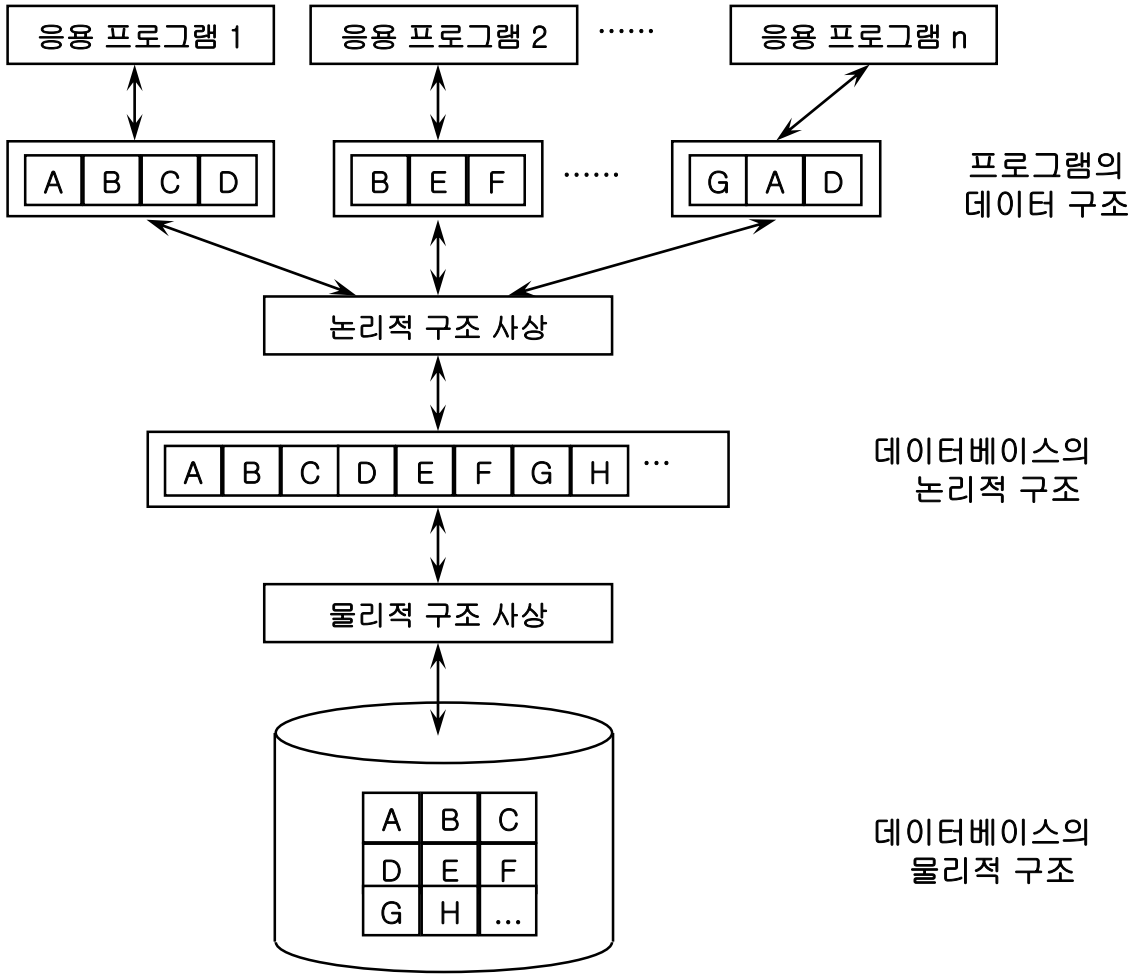
- 장점
  - 데이터 중복(redundancy)의 최소화
    - controlled (통제된) redundancy
  - 데이터의 공용(sharing)
  - 일관성(consistency) 유지
  - 무결성(integrity) 유지
  - 보안(security) 보장
  - 표준화(standardization) 용이
  - 전체 데이터 요구의 조정:상충되는 데이터가 있는 경우
- 단점
  - 운영비의 오버헤드
  - 특정 응용 프로그램의 복잡화: 서로 연관된 데이터의 처리
  - 복잡한 백업과 회복
  - 시스템의 취약성

데이터 독립성 (Independency)

- DBMS의 궁극적 목적: 데이터 독립성
  - 논리적 데이터 독립성
    - 응용 프로그램에 영향을 주지 않고 논리적 데이터 구조의 변경이 가능
    - 응용 프로그램의 효율적 개발 가능
  - 물리적 데이터 독립성
    - 응용 프로그램과 논리적 데이터 구조에 영향을 주지 않고 물리적 데이터 구조의 변경이 가능
    - 저장 장치의 효율적 개발
- 데이터 독립성 구현 기법
  - "Three level architecture" of database system
    - External schema, conceptual schema and internal schema
  - Mapping

3/12

Mapping & Data Independency



데이터베이스 관리 시스템의 역사

- 데이터베이스 랭킹
  - https://db-engines.com/en/ranking
- 초기 DBMS
  - Integrated Data Store (IDS)
    - 최초의 범용 데이터베이스 관리 시스템
    - 1960년대 초 General Electric의 Charles Bachman에 의해 설계
    - 네트워크 데이터 모델(network data model)의 기초
    - Bachman은 또한 데이터 구조도(data structure diagram)를 제안
  - IMS(Information Management System) DBMS
    - 1960년대 후반 IBM이 개발
    - 계층 데이터 모델(hierarchical data model)의 기초
    - 1970년대 초에는 대형 컴퓨터 회사들이 DBMS 제작 판매

관계형 DBMS

- 관계 데이터 모델(relational data model)
  - IBM의 E. F. Codd 가 1970년 제안
  - 데이터베이스 이론의 기초
  - 1980년대 DBMS의 주류가 되었고 계속 확장
- SQL (Structured Query Language)
  - IBM이 관계 DBMS의 일부로 개발
  - 세계 표준 데이터베이스 언어
- 하드웨어 독립적 운영 DBMS
  - Oracle, MySQL, MariaDB, PostreSQL
  - MS SQL Server, DB2, Ingres, Sybase, Informix

객체 지향 DBMS

- 사용자의 데이터베이스 응용에 대한 복잡성 증대
  - 데이터의 복잡한 분석을 지원하는 기능
  - 강력한 질의어, 멀티미디어 지원

- 사용자의 요구에 대처하기 위해 새로운 시스템 개발
  - 객체지향 DBMS(object-oriented DBMS, OODBMS)
  - 객체 관계 DBMS(object-relational DBMS, O-RDBMS)

NoSQL (Not Only SQL)

- 기존에 주로 사용하던 관계형 DBMS 탈피 노력
- 관계형 DBMS는 정규화(Normalization)를 통해 테이블을 잘게 분해
  - 일관성 (consistency) 유지
    - 아주 이상적인 구조 유지
  - 종속성과 중복성 없앰
  - 실제 사용할 때는 join 연산 필요 (시간이 많이 소요됨)
- NoSQL은 일관성을 조금 줄이더라도 성능 향상에 집중
  - 맛집 리스트와 평가 예제
    - 관계형 DBMS에서는 맛집 테이블과 평가 테이블 분리
    - NoSQL에서는 맛집 데이터에 평가들 같이 저장 가능
- 비정형 데이터, 읽기에 최적화된 데이터 등과 같이 유연성 제공
- Document (MongoDB), Key-Value, Graph, Wide Column 등